

实车挑战赛项目说明——轨迹规划框架篇

在这篇文档中，我们将聚焦于轨迹规划模块cav_traj_gen模块，并探讨如何开发。

一、代码框架

cav_traj_gen模块包含以下目录

```
| CMakeLists.txt
| Doxygen.config
| package.xml
|
|--include
|
|--launch
|   cav_traj_gen.launch
|
|--src
```

- include文件夹，包含一些基本的数学库、数据类型定义、数据池定义
- launch文件夹，包含有一个启动文件cav_traj_gen.launch
- src文件夹，存储了核心代码，包含GUI界面、轨迹规划等的实现。

```
└─src
    |   main.cpp
    |   |
    |   └─GUI
    |       |   gui.cpp
    |       |   gui.h
    |       |   GUI.pro.user
    |       |   gui.ui
    |       |   mainloop.cpp
    |       |
    |       └─widgets
    |           |   info.cpp
    |           |   └─3d_display
    |           |       └─battery
    |
    |   └─path_slicer
    |       |   path_slicer.cpp
    |       |   path_slicer.h
    |
    |   └─rosNode
    |       |   pub.cpp
    |       |   recv.cpp
    |       |   rosNode.h
    |
    |   └─traj_generator_template
    |       |   └─include
    |           |       opt.h
    |           |
    |           └─src
```

```
assit.cpp
opt.cpp
```

我们主要来看一下核心代码src文件夹。

- **GUI 文件夹**，实现基于Qt的图形化界面
- **path_slicer文件夹**，实现从全局参考路径截取局部参考路径
- **rosNode文件夹**，包含了ROS节点的定义与通信相关内容
- **traj_generator_template文件夹**，包含轨迹优化器的模板实现，**是你需要重点完成的部分**

二、数据结构

2.1 全局变量

在 `main.cpp` 中定义了两个核心全局变量，贯穿整个系统：

```
TRAJ_GEN TG;          // 轨迹生成器，包含所有核心组件
GUI_Set_S guiSet;     // GUI界面的设置参数
```

`TRAJ_GEN` 类（定义于 `include/globalDef.hpp`）将所有核心组件集成在一起：

成员变量	类型	说明
<code>TG.rosNode</code>	<code>ROSNODE</code>	ROS通信节点
<code>TG._rosh</code>	<code>ros::NodeHandle*</code>	ROS节点句柄
<code>TG.ssData</code>	<code>DataPool_S</code>	数据池（规划环境 + 统计信息）
<code>TG.opt</code>	<code>Opt</code>	轨迹优化器（ 核心，需要你完成 ）
<code>TG.path_slicer</code>	<code>Path_Slicer</code>	局部路径截取器

2.2 规划环境数据结构 `PlanningEnv_S`

定义于 `include/dataDefine.hpp`，包含规划所需的**全部输入信息**，共分为五部分：

部分	字段	说明
系统状态	<code>Estop</code> , <code>Go</code> , <code>Signal</code>	急停标志、启动标志、信号灯
第1部分：车辆状态	<code>x</code> , <code>y</code> , <code>heading</code>	车辆全局坐标与航向角
	<code>speed_x</code> , <code>acc_x</code> , <code>yaw_rate</code>	纵向速度、纵向加速度、横摆角速度
	<code>size_x/y/z</code> , <code>wheelbase</code>	车辆几何尺寸（长5m、宽2m、轴距2.85m）
第2部分：规划目标	<code>tf</code> , <code>xf</code> , <code>yf</code>	规划终点的时间、纵向、横向目标
	<code>xf_d</code> , <code>xf_dd</code>	终点速度与加速度目标
	<code>vd_yield</code> , <code>ad_yield</code>	控制层的期望速度/加速度上限
第3部分：障碍物	<code>obstacleVec</code>	障碍物列表（坐标、半径、类型等）
第4部分：局部参考路径	<code>refPathVec</code>	由 <code>path_slicer</code> 截取的局部路径点序列
	<code>NPN</code>	车辆当前位置对应的最近路径点索引
第5部分：全局参考路径	<code>whole_path</code>	完整的全局参考路径
	<code>cmd_num</code>	路径更新计数器（路径刷新时自增）
	<code>world_cx</code> , <code>world_cy</code>	世界坐标系原点偏置（UTM坐标）

注意： `x`、`y` 等坐标均为**相对坐标**（已减去 `world_cx/world_cy`），发布时需要加回偏置。

2.3 轨迹数据结构 Trajectory_S 与 Point_Xd

`Point_Xd`（定义于 `traj_generator_template/include/opt.h`）是路径点的数据结构，继承自 `X_Point`：

字段	说明
<code>x</code> , <code>y</code>	平面坐标
<code>t</code>	时间戳（相对时间，秒）
<code>v</code>	速度（m/s）
<code>heading</code>	航向角（rad）
<code>cr</code>	曲率（1/m）
<code>bound_left</code> , <code>bound_right</code>	左右边界宽度（m）
<code>x_d</code> , <code>x_dd</code> , <code>y_d</code> , <code>y_dd</code>	坐标一阶、二阶导数
<code>mileage</code>	路程（m）

`Trajectory_S` 是一条完整轨迹的数据结构：

字段	说明
<code>path</code>	<code>Point_Xd</code> 的向量，即轨迹点序列
<code>feasible</code>	轨迹是否可行
<code>Jcost</code>	代价函数值
<code>minDObj</code>	到障碍物的最小距离（m）
<code>minDbound</code>	到边界的最小距离（m）
<code>max_vx</code> , <code>max_acc_x</code> , <code>max_acc_y</code>	轨迹中的最大速度、纵向/横向加速度

三、各模块详解

3.1 程序入口： `main.cpp`

✔ 无需修改

程序的入口，完成以下工作：

- 1. 初始化ROS节点（节点名 `cav_traj_gen`）
- 2. 初始化Qt图形应用程序
- 3. 创建并显示GUI窗口，启动Qt事件循环mainloop
- 4. 程序退出时关闭ROS节点

3.2 ROS通信： `rosNode/`

✔ 无需修改

`ROSNode` 类负责所有ROS消息的收发。

订阅的话题（输入）：

话题名	消息类型	说明
<code>/Goal</code>	<code>cav_msgs::Goal</code>	全局参考路径及任务指令
<code>/VehicleState</code>	<code>cav_msgs::VehicleState</code>	车辆状态（位姿、速度等）

话题名	消息类型	说明
/Obstacle	cav_msgs::ObstacleVec	障碍物信息

发布的话题（输出）：

话题名	消息类型	说明
/PlannedPath	cav_msgs::PlannedPath	规划输出的轨迹

所有回调函数在接收消息后，会解析并写入全局数据池 `TG.ssData.goal`（即 `PlanningEnv_S`），供规划器使用。

3.3 路径截取：path_slicer/

✔ 无需修改

`Path_Slicer` 类从全局路径 `whole_path` 中，以车辆当前位置为中心，截取一段局部参考路径写入 `refPathVec`。

- 向后截取约 4m (`NPN - 4/interval` 个点)
- 向前截取约 50m (`NPN + 50/interval` 个点)
- 使用 `XM::find_NPN()` 查找车辆最近路径点索引 `NPN`

3.4 GUI主循环：GUI/mainloop.cpp

⚠ 需要你完成

这是整个系统的**主控制循环**，以 50Hz 定时触发。每个控制周期的执行流程如下：

1. 检查初始化状态 (`ini_flag`)
2. `ros::spinOnce()` 处理ROS消息
3. 检测路径更新 (`cmd_num`)，`path_slicer.get_local_reference_path()`截取局部参考路径
4. 设置规划参数 (速度/加速度约束等)，调用 `TG.opt.run()` 执行轨迹规划
5. `rosNode.publishPlanResult()` 对外发布结果
6. `update_info()` + `updateGL()` 刷新界面

在此文件中，以下规划参数会在每个周期传递给优化器：

参数	说明	默认值
<code>set_max_speed</code>	最大速度	8.0 m/s
<code>set_max_acc</code>	最大纵向加速度	4.5 m/s ²
<code>set_min_acc</code>	最大制动加速度	-3.0 m/s ²
<code>set_max_yawrate</code>	最大横摆角速度	5/2.4 rad/s
<code>set_max_acc_y</code>	最大横向加速度	2.0 m/s ²
<code>target_time</code>	规划时域（来自GUI中的滑动条）	5.0 s
<code>flag_checkObstacles</code>	是否检测障碍物（来自GUI）	true
<code>flag_ay_soft_const</code>	是否启用横向加速度软约束	true

3.5 可视化调试：GUI/widgets/3d_display/display_3d_mode1.cpp

💡 建议编辑，用于调试与验证规划效果

该文件实现了**俯视图（Bird's-Eye View）**的渲染（`view_mode = 1`），是调试轨迹规划算法最直观的工具。你可以在此随意添加或修改绘制内容，以可视化你的中间变量、候选轨迹、代价等，从而更快速地判断算法效果。与此对应，`display_3d_mode2.cpp`实现了**驾驶员视角**的渲染，视觉效果更酷炫，但对算法调试帮助有限，通常无需修改。

主渲染函数 `draw_mode1()`

每个GUI刷新周期（50Hz）均会调用此函数，按以下顺序依次绘制七类元素：

绘制顺序	内容	颜色	控制开关（ <code>guiSet</code> ）
1	坐标网格（1m间距）	灰色	<code>flag_Axis</code>
2	自车模型	—	始终显示
3	目标点（三角箭头 + 圆点）	浅绿色	始终显示
4	障碍物（圆形）	浅红色	<code>flag_Obstacle</code>
5	局部参考路径 （ <code>refPathVec</code> ）	蓝色	<code>flag_Ref_Path</code>
6	全局参考路径 （ <code>whole_path</code> ，含边界）	橙色	始终显示
7	最优规划轨迹 （ <code>traj_best</code> ，可行时）	黄色	<code>flag_BestPath</code>

通用绘制函数 `drawTraj()`

文件中提供了一个功能丰富的通用轨迹绘制模板函数，你可以直接调用它来可视化任何 `std::vector<T>` 类型的路径：

```
drawTraj(  
    &your_path_vec,    // 路径点向量指针（X_Point 或 Point_Xd 均可）  
    r, g, b,           // RGB颜色，范围 [0.0, 1.0]  
    gap,               // 采样间隔（1=逐点绘制）  
    line_width,        // 线宽（像素）  
    level,             // Z轴偏置（用于图层分层，避免重叠闪烁）  
    flag_point,        // 是否绘制离散点标记（圆形，直径0.15m）  
    flag_line,         // 是否绘制连续折线  
    flag_heading,      // 是否绘制各点航向箭头  
    flag_cr,           // 是否绘制曲率标记  
    flag_bound         // 是否绘制左右边界线  
);
```

如何添加自定义可视化

例如，若你在 `opt.cpp` 中生成了**一组候选轨迹** `TG.opt.candidate_trajs`，可在 `draw_mode1()` 中添加：

```
// 绘制所有候选轨迹（灰色折线）  
for (auto &traj : TG.opt.candidate_trajs)  
    drawOnePath(&traj);  
  
// 高亮最优轨迹（红色）  
drawTraj(&TG.opt.traj_best.path,  
    1.0, 0.2, 0.2, // 红色  
    1, 2.0, 0.5,  
    false, true, true, false, false);
```

只需确保你想绘制的数据已通过 `public` 成员变量暴露在 `TG.opt` 中即可。

3.6 轨迹优化器： `traj_generator_template/`

★ 核心，需要你重点完成

轨迹优化器由 `opt.h` 和 `opt.cpp` 实现，主要包含以下三个你需要完成的函数：

Opt() — 构造函数

在此初始化你的规划算法所需的内部数据结构和参数。

int Opt::ini(PlanningEnv_S *planning_env) — 初始化函数

将外部规划环境指针 `_env` 绑定，并在路径/任务更新时重置相关状态。每次路径更新（`cmd_num` 变化）后都会被调用。

void Opt::run() — 规划主函数（每50ms调用一次）

这是你实现轨迹规划算法的核心位置。函数结束后，框架会自动调用 `genControlPath()` 将结果重采样为控制层所需的 0.04s 间隔轨迹。

输入：通过 `_env` 指针访问所有规划所需信息（见第2.2节）

输出：将规划结果填入 `traj_best`

```
void Opt::run()
{
    if (_env == NULL) return;

    // TODO: 在此实现你的轨迹规划算法
    // 1. 读取车辆状态: _env->x, _env->y, _env->heading, _env->speed_x ...
    // 2. 读取局部参考路径: _env->refPathVec
    // 3. 读取障碍物信息: _env->obstacleVec
    // 4. 生成候选轨迹，评估代价，选出最优轨迹
    // 5. 将最优轨迹写入 traj_best.path, 并设置 traj_best.feasible = true

    // 框架已提供的示例（直接复制参考路径）:
    copy_generator();

    // 框架自动完成：重采样为控制时序
    genControlPath(&traj_best, TIME_GAP_CONTROL);
}
```

框架提供的辅助函数：

函数	位置	说明
<code>copy_generator()</code>	<code>opt.cpp</code>	示例：直接将参考路径前400点复制为轨迹
<code>genControlPath()</code>	<code>assit.cpp</code>	将规划轨迹样条插值为0.04s间隔，计算曲率和航向
<code>XM::find_NPN()</code>	<code>include/math/XM_path.h</code>	查找离车辆最近的路径点索引
<code>XM::update_dist()</code>	<code>include/math/XM_path.h</code>	计算路径各点的累计里程
<code>XM::cal_heading_by_2pts()</code>	<code>include/math/XM_path.h</code>	由路径点坐标计算各点航向角
<code>XM::cal_curvature_x()</code>	<code>include/math/XM_path.h</code>	由路径点坐标计算各点曲率

四、关键常量定义

定义于 `traj_generator_template/include/opt.h`：

常量	值	说明
<code>TIME_GAP_PATH</code>	0.25 s	规划轨迹的时间间隔
<code>TIME_GAP_CONTROL</code>	0.04 s	控制层轨迹的时间间隔（25Hz）
<code>MAX_ACCURACY</code>	0.1	数值精度阈值

定义于 `include/dataDefine.hpp`：

常量	值	说明
<code>FREQ</code>	10	系统基准频率（Hz）

这些关键常量请不要修改！

五、开发指南

5.1 你需要修改的文件

文件	优先级	说明
<code>src/traj_generator_template/src/opt.cpp</code>	☆☆☆ 必须完成	实现 <code>ini()</code> 和 <code>run()</code> 函数，即轨迹规划核心算法
<code>src/traj_generator_template/include/opt.h</code>	☆☆☆ 必须完成	在 <code>Opt</code> 类中定义你需要的成员变量和函数
<code>src/GUI/widgets/3d_display/display_3d_mode1.cpp</code>	☆☆ 建议修改	添加候选轨迹、中间变量等自定义可视化，便于调试（见3.5节）
<code>src/GUI/mainloop.cpp</code>	☆☆ 可能修改	调整规划参数、修改主循环逻辑
<code>src/GUI/gui.h</code> / <code>gui.cpp</code>	☆ 可选修改	根据需要增加GUI控件

5.2 开发规划算法的步骤建议

- 理解输入数据：**熟悉 `PlanningEnv_S` 中各字段的含义（见第2.2节）
- 运行示例：**先编译运行，确认 `copy_generator()` 能正常工作，了解整体数据流
- 实现规划算法：**在 `opt.cpp` 的 `run()` 函数中实现你的轨迹规划方法，例如：
 - 采样型方法：**在横向方向生成多条候选轨迹，碰撞检测后选出最优轨迹
 - 优化型方法：**以参考路径为基础，建立约束优化问题求解最优轨迹
- 填写输出：**将规划结果填入 `traj_best`，确保 `traj_best.feasible = true`，`traj_best.path` 中每个点的 `x, y, t, v` 字段不为空
- 框架自动处理：**`run()` 返回后，框架会自动调用 `genControlPath()` 完成航向角、曲率计算和时间重采样

5.3 注意事项

- 所有坐标（`x, y`）均为**相对坐标**，已减去世界原点偏置（`world_cx, world_cy`），计算时无需换算
- `traj_best.path` 中的时间 `t` 应为从当前时刻起的**相对时间**（秒），从 0 开始
- 若某周期无法生成可行轨迹，应将 `traj_best.feasible` 置为 `false`，控制层会据此判断是否制动
- `run()` 在50Hz主循环中调用，单次执行时间应控制在 **20ms以内**，避免阻塞
- 可通过 `printf()` 或 `ROS_INFO()` 打印调试信息，它们会输出到终端

六、launch文件说明

`launch/cav_traj_gen.launch` 用于启动本节点，支持以下参数：

参数	默认值	说明
output	screen	日志输出方式（screen 输出到终端，log 输出到文件）
required	true	节点退出时是否终止整个launch
respawn	false	节点崩溃后是否自动重启
target_time	1	规划时域初始值（秒）

启动命令：

```
roslaunch cav_traj_gen cav_traj_gen.launch
```